{EPITECH}

SUMMER SCHOOL_

< PART 5 - FULL STACK INTEGRATION />

SUMMER SCHOOL

< WEB DEVELOPMENT: TYING IT ALL TOGETHER />

> ”The whole is greater than the sum of its parts.”
> — **Aristotle**

The Case

Congratulations. You have built a dynamic Front-End (React) and a robust Back-End API (Node/Express). However, right now, they are living in two completely different worlds. Your newspaper is still displaying fake, hardcoded data, and the API is only being used by developers via Postman.

Furthermore, the Design Team has stepped in. They saw your Vanilla CSS and, frankly, they were not impressed. They want a modern, responsive, and beautiful UI, and they want it built using the industry standard: **Tailwind CSS**.

Your mission today is the ultimate Full-Stack challenge: You must connect your React application to your live API, and completely overhaul the visual design.

Project Requirements

You will learn how to connect your React application to your Back-End API, and how to use Tailwind CSS to create a beautiful, responsive design.

1. Feature: Live Data Integration (GET)

Your newspaper must display real data coming from your server.
- Start your Back-End server (`node server.js` on port 3000) and your React development server.
- Inside your React application, remove all hardcoded article arrays.
- Use the `useEffect` and `fetch` hooks to retrieve the articles from `http://localhost:3000/api/articles` when the application loads.
- **Acceptance Criteria:** When a user opens the React app, they should see the exact articles that are stored inside your Back-End's `articles.json` file.

2. Feature: The Journalist Dashboard (POST)

The journalists refuse to use Postman to publish articles. They need a user interface.
- Create a new component (e.g., `PublishForm.jsx`).
- It must contain a form with inputs for the Title, Excerpt, and Author.
- When the form is submitted, it must send a `POST` request to your API with the new data.
- **Acceptance Criteria:** Upon successful submission, the form should clear itself, and the new article should instantly appear in the article grid without requiring a manual page refresh.

3. Feature: The Visual Overhaul (Tailwind CSS)

The newspaper needs to look like a million-dollar tech startup.
- Install and configure Tailwind CSS in your Vite/React project.
- Delete your old `index.css` rules.
- Re-style your entire application using Tailwind utility classes (e.g., `flex`, `p-4`, `bg-slate-800`, `rounded-xl`, `shadow-lg`, `hover:bg-blue-600`).
- **Acceptance Criteria:** The application must look professional, modern, and be fully responsive (it should adapt cleanly if the browser window is resized).

Bonus (Optional)

If you finished early and want to impress the CTO, try implementing these advanced features:

- ✓ **Persistent Likes (PATCH):** Upgrade your ”Like” button. When clicked, it shouldn't just update the React state; it should send a `PATCH` request to your API so the ”Like” is permanently saved in the database!
- ✓ **Loading States:** Fetching data takes time. Add a spinning loader or a ”Skeleton” screen that displays while the React app is waiting for the Back-End to respond.
- ✓ **Error Handling:** What if the Back-End server crashes? Catch the error in your `fetch` request and display a beautiful error message to the user (e.g., *”Sorry, our servers are currently down. Please try again later.”*).

v1